



História e Evolução de Sistemas Computacionais

Integração de Sistemas - Aula #02

Prof. Dr. Carlos Melo

 carlos.melo@upe.br

1958



Antes da máquina, o computador era uma pessoa

- Na NASA dos anos 1960' o computador era um indivíduo bom de conta
- Várias mulheres calculavam a trajetória de objetos à mão, em turnos
- Em uma cena do filme Estrelas Além do Tempo, elas recebem um mainframe IBM 7090 e ninguém sabe operar
- Katherine Johnson e Dorothy Vaughan aprendem FORTRAN por conta própria para operá-lo



1969

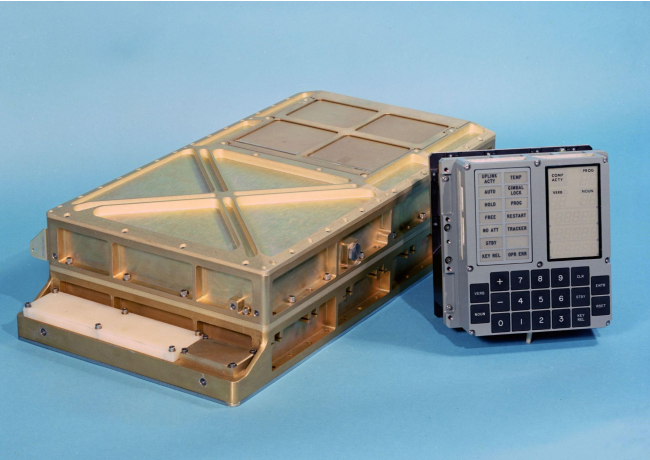


Quanta memória tinha no computador de bordo da Apollo 11?

- 4kb e uma CPU de 2MHz
- Pesava 32kg
- E era infinitamente inferior a um smartphone de 2010
- E ainda assim guiou uma nave até a Lua (e voltou)



carlos



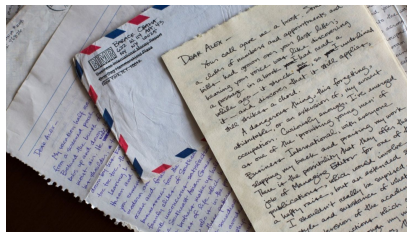
1975

Não existia:

- Internet
- Wi-Fi
- APIs REST
- JSON
- Docker
- Cloud
- Smartphones

Mas existia Tubarão

Como uma empresa compartilhava informações com outra empresa ou com um possível cliente?

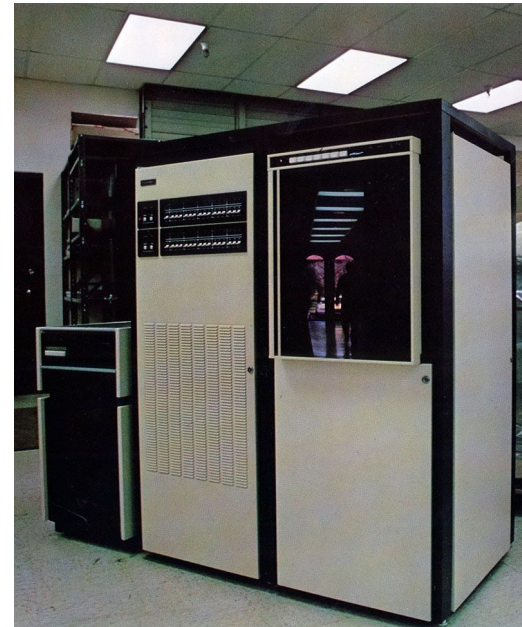




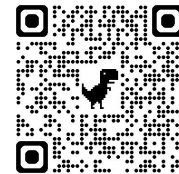
Como era um computador em 1975?

Ocupava uma sala inteira

- Um único computador era compartilhado por todos os funcionários
 - O acesso era feito por meio de um **terminal burro**
- Esse computador era conhecido como Mainframe e executava:
 - Financeiro
 - Folha de pagamento
 - Estoque
 - Contabilidade
 - RH
- O que aconteceria se esse computador parasse?



Como era um computador em 1975?



Os terminais burros não tinham:

- Disco, Processador, S.O...

Nada muito diferente de um...

- Chromebook
- sessão SSH
- navegador rodando Single Page Application (SPA)

O terminal burro ainda tá aí, só foi *gourmetizado*

Granted, our new ADM-3 is basic. Especially if you compare it with all the smart video terminals around (our ADM-1 or -2, for example). But the \$995 unit price puts it into a different perspective. As simple as it is, the ADM-3's one-card brain can help you move a lot of data. And it's compatible with most popular computers. That means, it fits all kinds of applications. Including yours.

Here's what you get for \$995: 12" diagonal screen. Full or half duplex operation at 11 selectable data rates. Bright, easy-to-read characters - 960 or 1920, displayed in 12 or 24 rows of 80 letters. 59 data entry keys arranged like on a typewriter which keeps operator training short and basic, too. What's more, our ADM-3 is alert. It says "beep" when you come to the end of a line. The rest of the time, it just keeps cool and quiet.

Our DUMB TERMINAL also offers you room for improvements. Its RS 232C interface extension port lets you hook up hard copy printer, magnetic tape recorder or additional (smarter) data terminals. And with a few options, you can make our ADM-3 answer back. Increase its vocabulary by adding upper and lower case. Transmit and receive independently selectable rates. Even enter just numbers on a numeric key pad.

After counting all its limited blessings, you have to admit one thing: you simply couldn't ask for more for \$995. At this low level, you can afford to order a dozen or more DUMB TERMINALS (and buy them at our even lower quantity discount price!).

DUMB TERMINAL. SMART BUY.
LEAR SIEGLER, INC.
ELECTRONIC INSTRUMENTATION
DIVISION DATA PRODUCTS
174 N. Brookhurst St.
Anaheim, CA 92803
Telephone (714) 774-1010

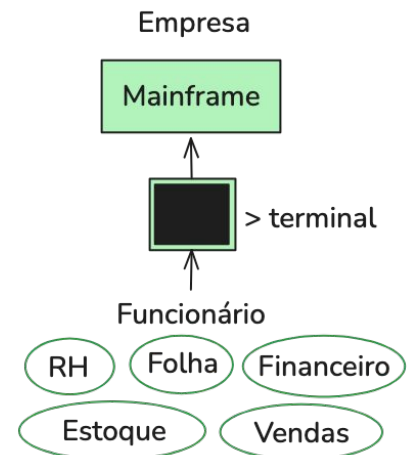


**INTRODUCING
THE \$995 DUMB
TERMINAL.**

Como era um computador em 1975?



- Nada para integrar
 - Um único programa, um único banco de dados e um único computador.
- Hoje a gente dá o bonito nome de **monolito** e muitos sistemas ainda são construídos assim, porque não é de todo ruim



carlos.melo@upe.br



Bom dia ☕



- Ponto único de falha (SPoF)
 - se o mainframe parasse a empresa parava
- O que acontecia com:
 - A folha de pagamento que fecha hoje?
 - O caminhão na doca esperando a nota fiscal?
 - A fila de 200 colaboradores em frente ao terminal para cadastrar suas tarefas?
 - O cliente no telefone que está aguardando para finalizar seu pedido?



O trabalho não era processado imediatamente



Hoje esperamos respostas em segundos (ou menos)

- Na década de 1970 essa ideia passava longe do padrão
- Uma tarefa poderia levar dias para ser concluída e a execução era sequencial
 - Modelo de Processamento em Lote (Batch)
- Na operação de computadores antigos a base era:

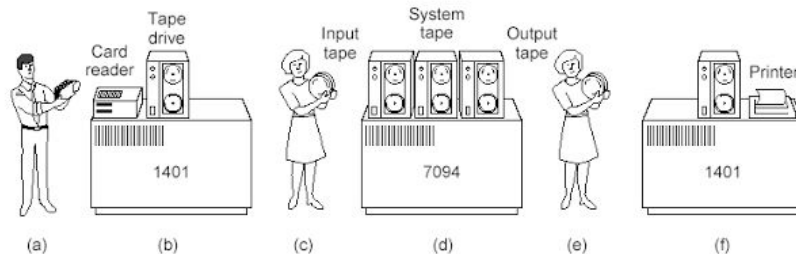
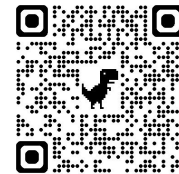


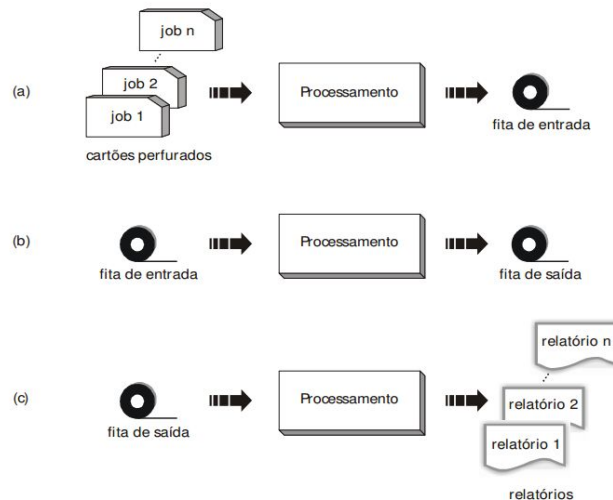
Figure 1-2. An early batch system. (a) Programmers bring cards to 1401. (b) 1401 reads batch of jobs onto tape. (c) Operator carries input tape to 7094. (d) 7094 does computing. (e) Operator carries output tape to 1401. (f) 1401 prints output.

O trabalho não era processado imediatamente



Era o modelo que fazia sentido para um recurso extremamente caro (milhões de dólares)

- Durante o dia o computador executava tarefas mais leves
- À noite, o processamento mais pesado (batch)
- Cada processamento era descrito por meio de um job:
 - JCL: Job Control Language
 - Qual programa executar
 - Quais arquivos serão utilizados
 - Onde salvar o resultado
 - Qual a próxima tarefa
- Hoje usamos *scripts* bash, actions, cron...



1985

Temos:

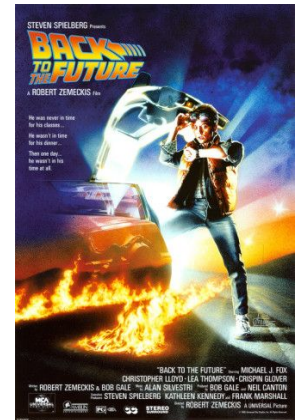
- PC de mesa
- Rede local, Ethernet
- Servidor de Arquivos
- Windows

Como que essa penca de computadores vai se comunicar entre si?



Mas ainda não temos

- Internet comercial
- HTTP
- JSON



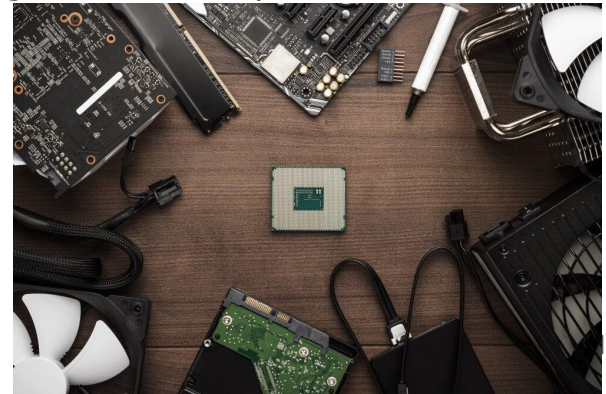


E a comunicação externa?

As empresas precisavam trocar informações com:

- Fornecedores
- Clientes
- Bancos
- Transportadoras
- Governos

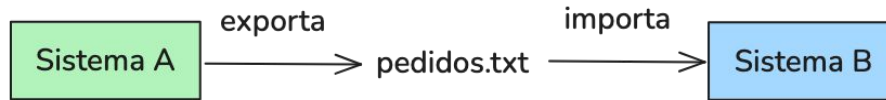
E sem rede compartilhada, sem protocolo em comum, sem garantia de que os dois lados usavam o mesmo fabricante de computador, dava ruim.





E a comunicação externa?

- Ao invés de conectar sistemas as empresas trocavam arquivos de texto
- Hoje enviamos uma requisição HTTP



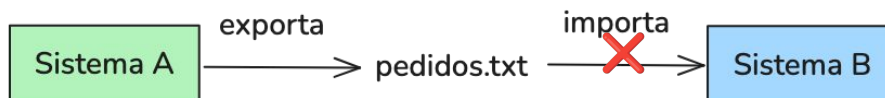
- A vantagem é que ninguém precisa estar no ar ao mesmo tempo.
- Uma distribuidora pode ler o arquivo às 6h da manhã com a loja que fez o pedido ainda fechada.



Mas ...

Cada empresa criava seu próprio formato de arquivo:

- Empresa A
 - 123;Notebook;10
- Empresa B
 - Notebook,123,10
- Empresa C
 - 123000010Notebook





Padronização

- As empresas perceberam que estavam separadamente resolvendo o mesmo problema
 - *E criando um novo*
- Surgiu então o EDI (Electronic Data Interchange)
 - Primeira grande padronização
 - ANSI X12 (1979) e UN/EDIFACT (1987)
- Ao invés de cada empresa criar seu formato todas usavam o mesmo padrão

```
1 ISA*00*          *00*          *ZZ*REMETENTE  *ZZ*DESTINO
   *260810*1430*U*00401*000000001*0*P*>
2  ST*850*0001
3  BEG*00*SA*PED-4471**20260810
4  P01*1*10*EA*3499.00**BP*123*VP*NOTEBOOK-X
5  CTT*1
6  SE*5*0001
7  IEA*1*000000001
```



Padronização

- Exemplos de EDIs:
 - Pedido de compra
 - Nota fiscal
 - Fatura
 - Aviso de entrega
- O EDI foi um conjunto de padrões para documentos comerciais
- Hoje uma empresa ou órgão publica uma API para que outras consumam



Padronização

- O EDI se popularizou por meio da pressão exercida pelos grandes
 - Montadoras e o varejo (Walmart nos anos 80–90) passaram a exigir EDI de quem quisesse ser fornecedor
 - Bancos padronizaram troca de arquivos de cobrança e pagamento
 - Governos passaram a exigir documentos fiscais em formato eletrônico



E como o arquivo chegava na outra ponta?

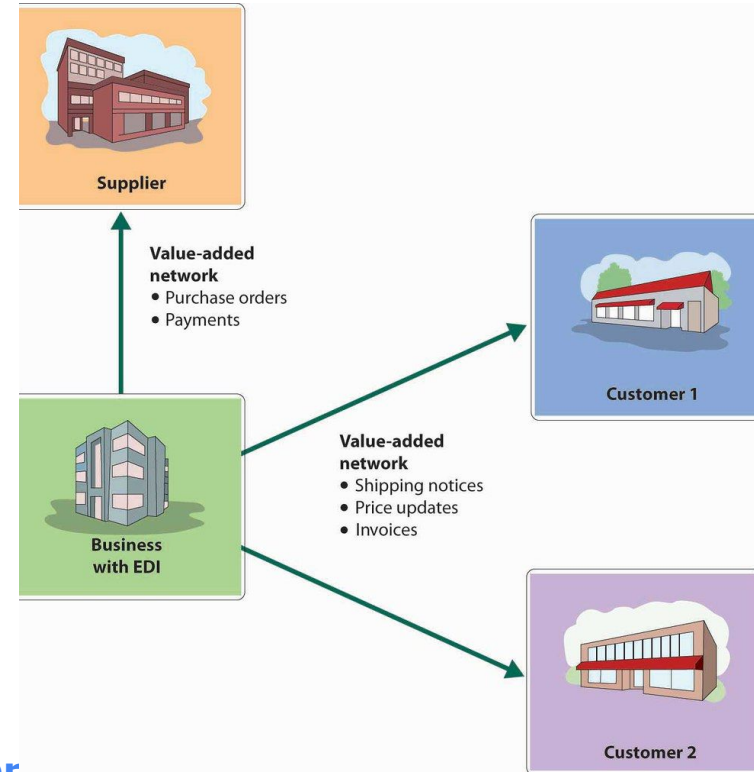
- Fitas magnéticas
- Disquetes
- Redes privadas (VAN)
- Linhas dedicadas
- FTP
- ...PPLPPC





Value Added Network (VAN)

- Um tipo de rede privada utilizada de forma exclusiva para troca de documentos
 - Transmissão segura
 - Confirmação de entrega
 - Armazenamento temporário
 - Comunicação entre empresas
- Porém as primeiras versões eram muito lentas, executar o pedido em uma VAN poderia levar dias
 - Até lá o próprio estoque já poderia ter acabado





Value Added Network (VAN)

- As empresas começaram a necessitar que a troca de informações passassem a ser:
 - Em tempo real
 - Com menos arquivos
 - Menos retrabalho
 - Menos erros
 - De forma direta



Certo... e tudo isso foi pra cova?

- IBM z16 (2022) continua sendo vendido e a linha z/OS tem sucessor ativo
- A maior parte do processamento de cartão e de core bancário do mundo ainda passa por mainframe e COBOL
- Grandes bancos brasileiros mantêm core em mainframe, com uma camada de APIs por cima

```
x3270-2 ZeusZVM
File  Options
z/VM 5.1 ONLINE
This is Zeus, the mainframe of the
IBM zSeries University Program for Europe

zSeries
Europe
University
System
ZEUS

Access only permitted when authorized by explicit agreement with IBM

To access the Zeus z/OS system, type DIAL ZOS
To access your own university's second-level z/VM system,
type DIAL followed by its name, for example: DIAL F00VM

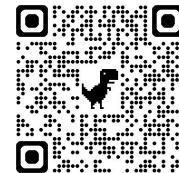
System ==>
RUNNING ZEUSZVM
023/015
```



Exercício - Lendo um pedido

- Uma distribuidora manda uma especificação em uma largura fixa
 - 123456789012345678901234567890
 - posição 1-6: código do produto
 - posição 7-10: quantidade
 - posição 11-30: nome do produto

```
1 $ cat dados/pedidos_20260811.txt
2 00PEDIDOS20260811
3 123 0010NOTEBOOK
4 456 0003MOUSE
5 789 0025TECLADO
6 99000003
```

Exercício - Lendo um pedido

- Implemente `parse_linha(linha)`
 - Recebe: uma `str` com uma linha do arquivo
 - Devolve: dict com as chaves `codigo (str)`, `quantidade (int)` e `produto (str)`
 - Devolve `None` se a linha não puder ser aproveitada
- Implemente `ler_pedidos(path)`



```
1 def parse_linha(linha: str) -> dict | None:  
2     raise NotImplementedError
```



```
1 def ler_pedidos(path: str) -> list[dict[str, int]]:  
2     raise NotImplementedError
```

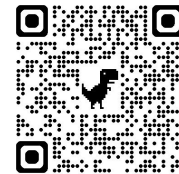


Exercício - Lendo um pedido

- Regras
 - Espaço à direita é preenchimento, não faz parte do dado
 - Linha com menos de 30 caracteres é inválida
 - código e produto em branco tornam a linha inválida
 - quantidade que não for numérica torna a linha inválida
 - quantidade igual a zero é válida
- Resultado Esperado

```
1 $ cat dados/pedidos_20260811.txt
2 00PEDIDOS20260811
3 123 0010NOTEBOOK
4 456 0003MOUSE
5 789 0025TECLADO
6 99000003
```

```
1 ler_pedidos("pedidos.txt")
2
3 [{'codigo': '123', 'quantidade': 10, 'produto': 'NOTEBOOK'},
4 {'codigo': '456', 'quantidade': 3, 'produto': 'MOUSE'},
5 {'codigo': '789', 'quantidade': 25, 'produto': 'TECLADO'}]
```



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br